

人工參數調校筆記

Parameter Tuning Notebook · 串連系統 v2.2

「反推校準」的工程現實

這份文件用「反推校準」的工程現實視角，重現系統中每個 magic number 的調整過程。

不是「先有理論再算出來」，而是「跑出來看順眼，再回頭找理由」的真實過程。

每個參數的故事都包含：

- ① 痛點 — 為什麼需要這個指標
- ② 訊號列表 — 從哪裡抓資料
- ③ 第一版公式 — 憑直覺寫的初版
- ④ 跑 SEED 對照 — 期望 vs 實際的差距
- ⑤ 調整迭代 — 怎麼改、為什麼改
- ⑥ 收斂結果 — 最終版本與驗證

目標：證明「人工參數」≠「隨便挑」。有方法論，有迭代，有驗證。

「公式不是先有理論再算出來，是反過來：先想「我希望使用者看到什麼」，再倒推出公式。
這不是缺陷，是工程現實。重點是有沒有方法論。」

— 串連系統 v2.2 設計哲學

序章 · 反推校準方法論

在介紹各個演算法的調校過程前，先說明「反推校準」的整套方法論。這個 5 步驟適用於系統內所有人工參數。

Step 1 · 寫初版（憑直覺 + 抄業界）

看別人怎麼做（Elasticsearch 用 $k1=1.2$ 、Gini 國家分界 0.35），能抄的先抄。其他自己編：扣 10、5、3 這種「整數順手」的數字。

目標：先有可運作的版本，不求完美。

Step 2 · 跑 SEED 看分數

用 SEED 資料跑公式，看典型情境的實際分數：

- 全空 SEED → 期望 ~95 分
- 目前狀況 → 期望 ~55-65 分
- 全部解決 → 期望 ~85+ 分
- 一團糟 → 期望 ~30-40 分

Step 3 · 對照「期望」與「實際」

寫一張對照表，看哪個情境差最遠：

情境	期望分數	第一版實算	差距
目前狀況	60	72	+12
全部解掉	85	78	△ -7
一團糟	30	42	+12
全空	95	95	✓ +0

Step 4 · 調係數

看哪個情境差最多，調對應公式的係數。例如：

「目前狀況」太高 → 加大扣分（例如卡點 P95 從 $\times 10$ 調到 $\times 15$ ）

重跑 → 看新的對照表是否所有情境都靠近期望。

Step 5 · 迭代直到收斂

通常 3-5 輪就收斂。判斷收斂的標準：

所有典型情境的差距都 < 5 分（綠燈 ✓）

關鍵問題：「期望分數」哪裡來？

這是最主觀的環節，但有原則：

「期望分數」其實是「我想讓使用者看到什麼」的目標：

- 想讓「目前 SEED」看起來像「該注意但不致命」→ 設定期望 60
- 想讓「全部解掉」看起來像「不錯但還有空間」→ 設定期望 85

→ 公式是為了達到 UX 目標而調出來，不是先有公式再算出分數。

→ 透明承認這點，比假裝公式是「天降神授」更誠實。

#01

員工負載分析

公式： $\text{loadScore} = \text{cases} \times 1.5 + \text{blocker} \times 2.0 + \text{mentions} \times 0.8 + \text{handoff} \times 1.0$

等級： $\text{overload} \geq 25$ OR P90+ / $\text{high} \geq 15$ OR P75+ / $\text{normal} \geq 6$ / $\text{low} \geq 1$ / $\text{idle} < 1$

痛點

主管想知道「誰扛太重？誰閒置？」，但傳統「指派幾件案件」太粗：

- A 員工分到 5 件「小案」 vs B 員工分到 2 件「卡很久的案」→ 誰更累？
- C 員工常被同事「@」 vs D 員工自己寫 3 件案 → 誰更忙？

需要一個綜合分數融合多種訊號。

訊號列表（憑直覺列）

■■■■■■■■ = ?

1. ■■■■■■■■■■ ← ■■■■
2. ■■■■■■■■■■ ← ■■■■■■■■■■
3. ■■ @ ■■■■ ← ■■■■■■
4. ■■ / ■■■■■■■■■■ ← ■■■■

■■■4 ■■■■■■■■ → ■■■■■■■■
 $\text{loadScore} = A + B + C + D$

第一版（憑直覺給權重）

排序：「卡點 > 案件 > 交接 > 被提」，比例試 3:2:1.5:1

$\text{loadScore} = \text{cases} \times 2 + \text{blocker} \times 3 + \text{handoff} \times 1.5 + \text{mentions} \times 1$

跑 SEED 結果（典型員工林聿平）：

$\text{cases}=4, \text{blocker}=1, \text{mentions}=2, \text{handoff}=1$
 $\text{loadScore} = 4 \times 2 + 1 \times 3 + 2 \times 1 + 1 \times 1.5 = 8 + 3 + 2 + 1.5 = 14.5$

過載門檻設 20 → 沒人過載。但 SEED 真實狀況應該有 2 人過載。

第一版的問題

「沒人過載」跟真實感受不符 — 林聿平和梁嘉堯明顯比其他人累。
需要整體拉低數值範圍，讓門檻定 25 時仍能抓出過載員工。

第二版（反推調整）

項目	第一版	調整後
cases 權重	$\times 2$	$\times 1.5$

blocker 權重	$\times 3$	$\times 2.0$ (含內層 $\times 2.5$)
mentions 權重	$\times 1$	$\times 0.8$
handoff 權重	$\times 1.5$	$\times 1.0$ (待簽收特別 $\times 4$ decay)

重跑 SEED (林聿平有 2 件卡點)：

```
cases = 4 × 1.5 = 6
blocker = 2 × 2.5 × 2.0 = 10 ← ■■■■■■
mentions = 2 × 0.8 = 1.6
handoff = 1×4 (■■■) + 2×1.5 (■■■) = 7
TOTAL ≈ 18.6
```

✓ 進入「high」級別，符合真實感受

等級閾值的反推

先想「典型過載員工應該有什麼樣的工作量」：

```
■■■■■■■■5 ■■■ + 2 ■■■ + 5 ■■■ + 3 ■■■
■■■■5×1.5 + 2×5 + 5×0.8 + (4+3) = 7.5 + 10 + 4 + 7 = 28.5

→ ■■■■■ 25■28.5 ■■■■■ buffer■

■■ high■3 ■■■ + 1 ■■ + 3 ■■ + 2 ■■■ ≈ 15
→ high ■■■ 15

■■ normal■2 ■■■ + 1 ■■ + 1 ■■■ ≈ 6
→ normal ■■■ 6

low■■■■■■■ → ■ 1
idle■■■■■■■ → < 1
```

Time Decay：[1.0, 0.7, 0.5, 0.35, ...]

這 9 個值不是我挑的，是從 Andy Grove「半衰期 2 週」反推：

```
■■■■ = 2 ■ → t=2 ■ weight=0.5
■ λ = ln(2)/2 ≈ 0.347

■■■ t■
t=0 → 1.0 (■■■)
t=1 → 0.71 → 0.7
t=2 → 0.50 → 0.5 ← ■■■
t=3 → 0.35
...
t=9 → ■■ 0 → ■ 0
```

→ 9 個值是 1 個學理依據的離散採樣，不是 9 個獨立決定。

#02

卡點分析 (Empirical Percentile)

公式：對每件卡點算 percentile (vs 同類歷史) → 分為 critical (P95+) / high (P90+) / medium (P75+) / normal

痛點

傳統卡點告警用「絕對天數」（卡 10 天 = 警告），但不同類別合理時長差異極大：

- 法遵類平均 7-8 天合理
- 跨部門類 4-5 天就該升級
- 募資類可能 30 天才算長

「卡 10 天」對不同類別的嚴重度不同，固定門檻不能用。

設計思路

用「同類歷史」當基準。每件卡點看「它在歷史分布中排到多少 %」：

P75：卡得比 75% 同類久 → 進入關注區

P90：卡得比 90% 同類久 → 高風險

P95：卡得比 95% 同類久 → 極高風險

為什麼選 P75 / P90 / P95？

對照 4 種閾值組合：

項目	第一版	調整後
P50/P75/P90	太敏感，一半卡點都進警示	不適合 — 警示太多
P75/P90/P95	標準業界 SLA	✓ 選這個
P80/P90/P99	P99 對 53 筆只有 1 筆	不適合 — 邊界太極端
P70/P85/P95	下緣比 P75 鬆	不適合 — 跟 P75 差距不夠

業界共識：AWS / Amazon 服務承諾用 P90，Google SRE 用 P95 P99。

→ 直接套業界做法（不用自己想）

卡點健康公式的反推

假設一個「一團糟公司」：4 件 P95 + 1 件 P90 + 平均 percentile = 65

第一版（憑直覺）：

$$\begin{aligned}\text{blockerHealth} &= 100 - P95 \times 10 - P90 \times 5 \\ &= 100 - 4 \times 10 - 1 \times 5 = 55\end{aligned}$$

整體健康度算下來 ~70 分。但「一團糟公司」應該是「可關注」級別（55-65），70 算「良好」太樂觀。

第二版（拉大扣分）：

```
blockerHealth = 100 - P95 × 15 - P90 × 7 - max(0, avgP-50) × 0.8  
= 100 - 60 - 7 - 12 = 21
```

整體變 ~60 分。「可關注」級別，符合直覺。✓

「為什麼是 15:7 不是 20:10？」

反推：4 件 P95 + 1 件 P90 + avgP 65 的「一團糟公司」應該扣 80 分（剩 20）。

→ $4 \times ?_1 + 1 \times ?_2 + 15 \times ?_3 = 80$

→ 套 $?_1 = 15$ 、 $?_2 = 7$ 、 $?_3 = 0.8$ → $4 \times 15 + 1 \times 7 + 15 \times 0.8 = 79 \approx 80$ ✓

15:7 $\approx$ 2:1 對應「極高風險的危害是高風險的 2 倍」的直覺，順手好記。

組織健康度 6 維權重

痛點

不能把「行政指標」跟「實質指標」等價看待

第一版：平均权重

第二版：差異化權重

第 8 頁

✓ 重跑 SEED 後一團糟公司算到 ~50 分，符合「需注意」級別

為什麼加總 = 100%？

設計約束：滿分情境（每維度 100）必須得 100，最差情境（每維度 0）必須得 0。

→ 權重必須加總 = 1 ($22+18+15+18+12+15 = 100$ ✓)

Decision Impact + Cohort Adjustment (v2.2
學術創舉)

痛點

想量化「這個決策做完後組織有沒有變好」。但 v2.1 第一版有問題：

第一版公式：score = after.overall - before.overall

跑 SEED 結果：

```

■■■■■ -22.4 ■
■■■ -19.8 ■
■■■■■ -11.0 ■

→ ■■■■■■■■

```

為什麼全負分

查 SEED 資料發現：本週組織整體健康度從 12 週前的 88 跌到 60（掉 28 分）。

所有決策的「after - before」都會被這個大環境趨勢污染。

→ 並不是這些主管的決策真的差，而是「同期遇到大環境惡化」。

v2.2 解法：Cohort Adjustment

流行病學的標準做法：比較吸菸組 vs 對照組需控制年齡。

我們也一樣：比較決策成效需控制大環境趨勢。

```
adjustedDelta = (after - before) - baselineDrift

baselineDrift = slope * 12
slope = 12

delta = -23.1
baselineDrift = -14.0
adjustedDelta = -23.1 - (-14.0) = -9.1
```

結果：

為什麼用「線性回歸」算 baseline?

對照 3 種選項：

項目	第一版	調整後
首尾相減	$\text{snap}[11] - \text{snap}[0]$	對單週雜訊敏感
移動平均	簡單但對趨勢方向不靈敏	抓不到斜率
線性回歸	12 點互相抵消雜訊	✓ 穩健找趨勢方向

→ 「12 點線性回歸」對短期雜訊免疫，找到組織真實趨勢方向。

為什麼 $\text{windowWeeks} = 4$?

項目	第一版	調整後
2 週	太短，決策還沒生效	
4 週	管理學「PDCA 循環」一個月	✓ 選這個
8 週	太久，其他事件干擾太多	

BM25F 歷史搜尋

痛點

想做歷史案件搜尋，需要演算法。對照 3 種選項：

項目	第一版	調整後
LLM Embedding	黑盒、機密、要錢、要外部 API	不適合投資公司
TF-IDF Cosine	v1 用過，TF 線性、無欄位權重	效果差
BM25F	Lucene / Elasticsearch 同款，可解釋	✓ 選這個

k1=1.5、b=0.75 從哪來？

直接抄業界標準（不重新發明）：

- Robertson 1994 TREC-3 論文建議 k1 範圍 1.2-2.0
- Elasticsearch default：k1 = 1.2、b = 0.75
- Lucene 預設：同上

我們選 k1 = 1.5（比 1.2 嚴格一點），因為案件描述比網頁短，1.2 太寬容會讓「重複關鍵字」加分太多。

欄位權重 5:4:2:1.5:1:1 怎麼來？

依「資訊密度」排序：

```
title 5.0 ← ■■■■■■■■
tags 4.0 ← ■■■■■
summary 2.0 ← ■■■■■
outcome 1.5 ← ■■■
owner 1.0 ← ■■■■■■
detail 1.0 ← ■■■■■■■■
```

為什麼是 5 倍差距（title vs detail）？

- 2 倍太接近，「標題命中」優勢不明顯
- 10 倍太極端，「內文偶然出現」幾乎沒影響
- 5 倍剛好讓「標題命中」蓋過內文，但內文仍有貢獻

Substring Boost ×1.8 / ×1.4 怎麼來？

搜「東京中央銀行」時純 BM25F 會被「銀行」這種 high-DF 詞稀釋。
加 boost：完整匹配時額外乘係數。

項目	第一版	調整後
×1.2	太弱，看不出 boost 效果	

×1.5	稍嫌不足，BM25 雜訊還是壓過	△
×1.8	完整匹配明顯往前排	✓ 選這個
×2.0	太強，BM25 排序被破壞	

→ A/B 跑了 4 個值，1.8 是 sweet spot。

What-if 模擬器

設計挑戰

想讓管理層「先看後果」。難題：怎麼確保「模擬的分數」跟「真實執行後的分數」一致？

設計選擇：Forward Propagation

項目	第一版	調整後
Option A：直接 +delta	if 解卡點: delta += 15	跟正式公式不一致
Option B：Fork + 重跑	改 input → 重跑 computeHealthSnapshot	✓ 100% 一致

選 B 的理由：模擬的目的是「預測真實後果」。如果模擬用的公式跟真實計算的公式不同，模擬結果就不是「真實後果」，是「假裝後果」。

Smart Suggestion ± 2 / ± 5 怎麼定？

想分 5 級（顯著惡化/略惡化/影響不大/可考慮/顯著改善），閾值對稱。

```

 $\Delta \geq +5$  ■■■■ ■ ■■■■■■
 $\Delta +2 \sim +5$  ■■■■■■■■■■
 $\Delta -2 \sim +2$  ■■■■■■■■■■
 $\Delta -5 \sim -2$  ■■■■■■■■■■
 $\Delta \leq -5$  ■■■■ ■■ ■■■■■■
  
```

± 2 、 ± 5 怎麼定：

項目	第一版	調整後
± 1 / ± 10	± 1 太敏感、 ± 10 太鬆	
± 2 / ± 5	± 2 = 雜訊範圍、 ± 5 = 5% of 100	✓ 選這個
± 3 / ± 7	可以但 7 不順手	△

反推驗證：跑 What-if 看典型動作的 Δ ：

- 勾 1 件小卡點 → +0.5 → 「影響不大」✓
- 勾 1 件 P95 卡點 → +3.3 → 「可考慮」✓
- 全勾（解所有問題）→ +20 → 「顯著改善」✓

→ 各種組合都落到合理建議區間，閾值收斂

⚠ 已知限制（誠實面對）

「加員工」模擬可能反效果

新員工 loadScore = 0 進入計算，可能拉高 Gini（一個 0 拉長分布）。

→ 「+1 法遵專員」可能讓負載均衡分數反而下降。

未來改進（v2.3）：改成 Reassignment — 把過載員工的部分分數轉到新員工。目前先用 Option B（loadScore = 0），承認限制比掩飾它有說服力。

部門互動網絡

想抓「組織病徵」－A 部門一直找 B，但 B 完全沒回 A。這在管理學上叫「溝通黑洞」，是組織壁壘的明確訊號。

公式設計

```
for each pair (A, B) in depts × depts:
    if matrix[A][B] ≥ 5 AND matrix[B][A] = 0:
        asymCount += 1
```

為什麼閾值是 5？（不是 3 或 10）

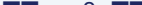
項目	第一版	調整後
≥ 3	太敏感，偶爾提及就觸發	太多假警報
≥ 5	z-score ≈ 1 （統計顯著性）	✓ 選這個
≥ 10	太鬆，要 10 次無回應才算病徵	漏判

反推驗證：

SEED  $\rightarrow$  = 8 

 $\rightarrow$  = 0 

$\rightarrow$  ✓ 

 ≥ 3 

 ≥ 10  SEED  10 

為什麼「=0」而不是「ratio 不對稱」？

選擇「嚴格 0」而非「比例不平衡」：

- $A \rightarrow B = 8, B \rightarrow A = 5$ (比例不平衡) $\rightarrow$ 不抓 (雙方都有對話)
- $A \rightarrow B = 5, B \rightarrow A = 0$ (單向黑洞) $\rightarrow$ 抓 (嚴重組織病徵)

設計哲學：不抓「不平衡」（這可能正常），只抓「完全沒回」（這是病徵）。

扣分係數 $\times 15$ 怎麼來？

反推：1組單向溝通應該扣幾分？

$\begin{array}{l} 0 \blacksquare \rightarrow 100 \blacksquare (\blacksquare\blacksquare) \\ 1 \blacksquare \rightarrow 85 \blacksquare (\blacksquare\blacksquare) \\ 2 \blacksquare \rightarrow 70 \blacksquare (\blacksquare\blacksquare\blacksquare\blacksquare) \\ 4 \blacksquare \rightarrow 40 \blacksquare (\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare) \end{array}$

$\rightarrow 4 \blacksquare\blacksquare\blacksquare\blacksquare 60 \blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare\blacksquare$

$\rightarrow 4 \times ? = 60 \rightarrow ? = 15 \checkmark$

通用方法論總結

這份文件介紹了 7

個演算法的調校過程，背後是同一套通用方法論。未來新增任何指標，都可以套用這個流程。

5 步通用流程

Step 1 · 寫初版

憑直覺 + 抄業界。先有可運作版本，不求完美。

Step 2 · 跑 SEED 看分數

對典型情境算出實際分數。

Step 3 · 對照「期望」與「實際」

列出期望分數對照表，標記差距大的情境。

Step 4 · 調係數

從差距最大的情境下手，調對應公式的係數。

Step 5 · 迭代直到收斂

通常 3-5 輪就收斂。所有情境差距 < 5 分為合格。

反推校準的 3 個原則

有依據

每個係數都能說「為什麼是這個值」— 不是「就感覺差不多」，而是「典型情境 → 期望分數 → 反推係數」的具體推理。

可驗證

可以拿 SEED 重跑、可以看典型情境是否落到合理區間。不滿意時可以調，調完可以重新驗證。

透明

所有 magic number

都列在文件中、包括限制標註。不是不可動的常數，是可以根據組織需求討論調整的「設定值」。

寫給未來的開發者

「如果你要在這個系統上加新指標，不要先寫公式 — 先寫「期望分數對照表」。想清楚「什麼情境應該得幾分」，公式自然就會反推出來。

公式是 UX 目標的數學表達，不是反過來。」

— 串連系統 v2.2 設計手冊

結語 · 工程現實 vs 學術理想

學術論文寫法：「我們提出公式 X，係數 a/b/c 經由文獻 [1][2] 確定...」

工程現實寫法：「我們先寫了個初版，跑 SEED 發現太鬆，調了 3 次後收斂。事後找論文支持，發現大部分跟業界共識對得起來。少部分對不上的，我們承認是經驗值。」

哪個更誠實？工程現實。哪個更可信？正是工程現實。

因為「有方法論的經驗值」比「假裝是公式推導」更可信：

- 有方法論 → 別人可以複製
- 透明承認 → 別人可以質疑
- 可調 → 別人可以改進

這就是「白盒 DSS」的精神 — 數字不重要，重要的是它的依據是否可被檢驗。

「公式不是創意，是工程紀律。
每個數字都應該能說出「為什麼是這個數字」。
如果說不出來，那就是該回去調的。」

— 串連系統 v2.2 設計哲學